



**Sunbeam Institute of Information Technology**  
**Pune and Karad**

## **Data structures and Algorithms**

Trainer - Devendra Dhande

Email – [devendra.dhande@sunbeaminfo.com](mailto:devendra.dhande@sunbeaminfo.com)

- Solving a polynomial (using linked list)

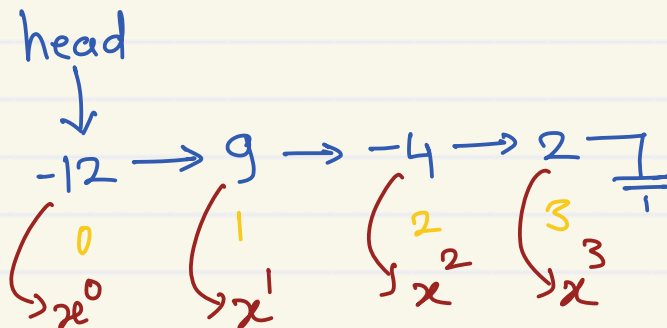
$$f(x) = 2x^3 - 4x^2 + 9x - 12$$

$$f(x) = 2x^3 - 4x^2 + 9x^1 - 12x^0$$

$$\text{if } x=2$$

$$\begin{aligned} f(2) &= 2 \times 2^3 - 4 \times 2^2 + 9 \times 2^1 - 12 \times 2^0 \\ &= 16 - 16 + 18 - 12 \\ &= 6 \end{aligned}$$

- represent polynomial in the form of linked list of coefficients



$$\begin{aligned} -12 \times 2^0 &= -12 \\ 9 \times 2^1 &= 18 \\ -4 \times 2^2 &= -16 \\ 2 \times 2^3 &= 16 \end{aligned}$$

6 ← result

```

int solvePolynomial(Node trav, int x) {
    int res = 0;
    int px = 1;
    while (trav != null) {
        int coef = trav.data;
        res += coef * px;
        trav = trav.next;
        px = px * x;
    }
    return res;
}

```

# Stack and Queue using linked list

## Stack (LIFO)

1. Add First  
Delete First

2. Add Last  
Delete Last

- while choosing operations of linked list ensure that time complexity will be  $O(1)$

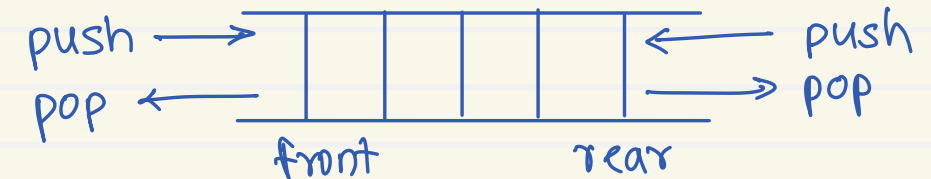
## Queue (FIFO)

1. Add Last  
Delete First

2. Add First  
Delete Last

## Deque (Double Ended Queue)

- data can be inserted and removed from both the ends (front & rear).



- can be implemented with the help of circular array or linked list.

- deque do not follow "FIFO" principle

### Operations :

1. pushFront

2. popFront

3. peekFront

4. pushRear

5. popRear

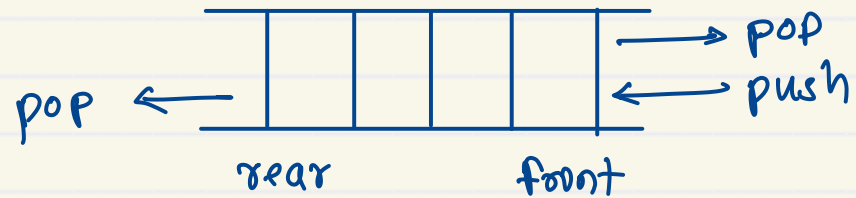
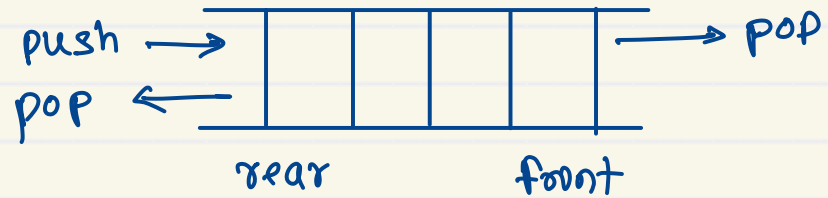
6. peekRear

7. isEmpty

8. isFull

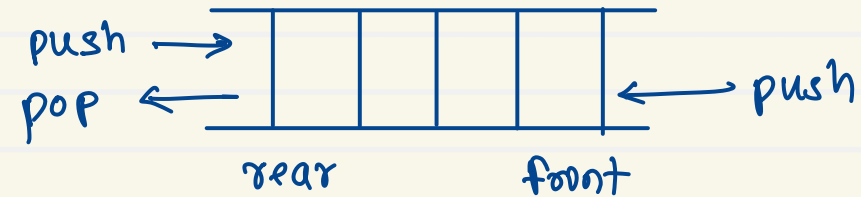
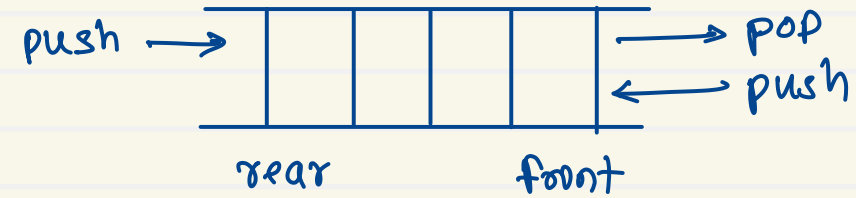
(in case of array)

## Input Restricted Deque



( push ) input - allowed from only one end  
( pop ) output - allowed from both the ends

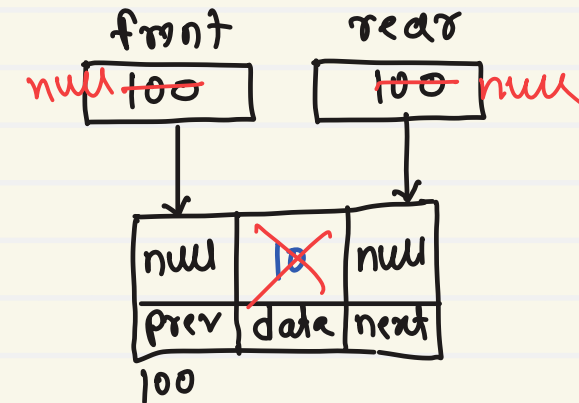
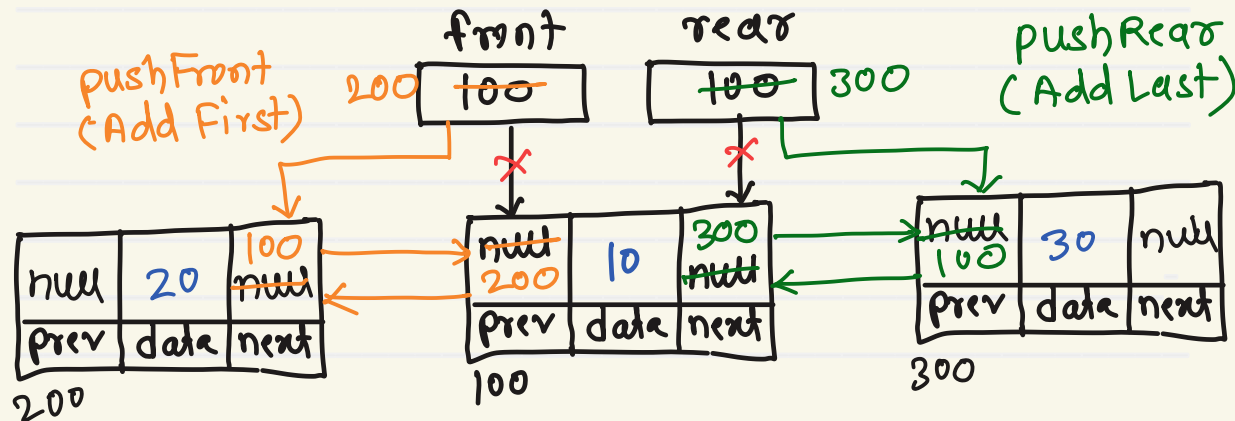
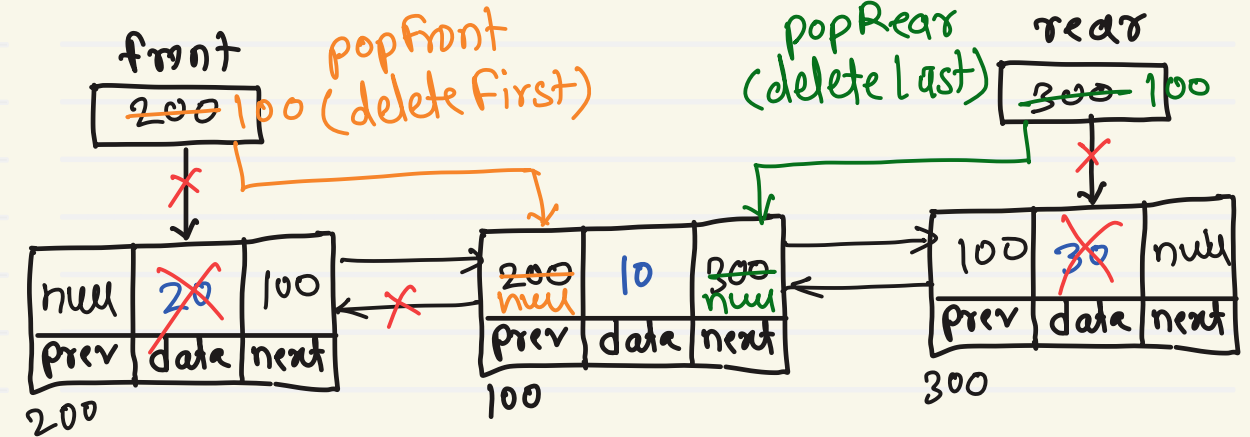
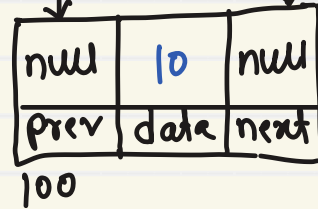
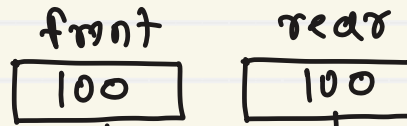
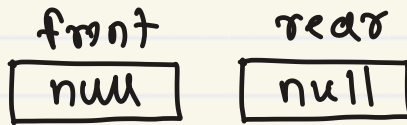
## Output Restricted Deque



( pop ) output - allowed from only one end  
( push ) input - allowed from both the ends

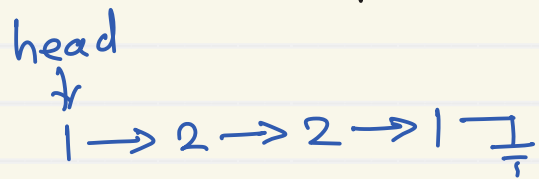
# Deque using Linked list

head  $\rightarrow$  front (addFirst, deleteFirst)  
tail  $\rightarrow$  rear (addLast, deleteLast)



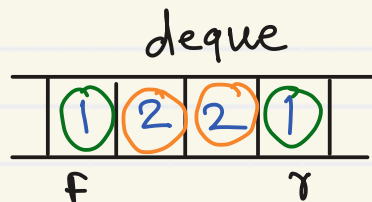
1. to implement multiple stack or queue

2. to check palindromes



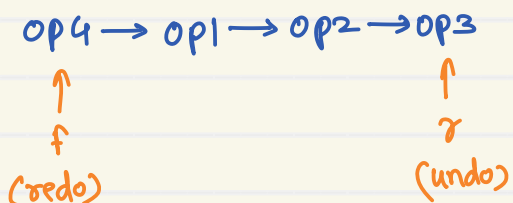
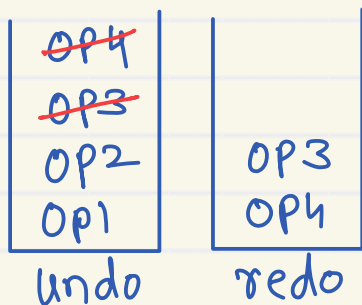
$$\text{itrs} = n + n/2 = \frac{3}{2}n$$

$$T(n) = O(n)$$



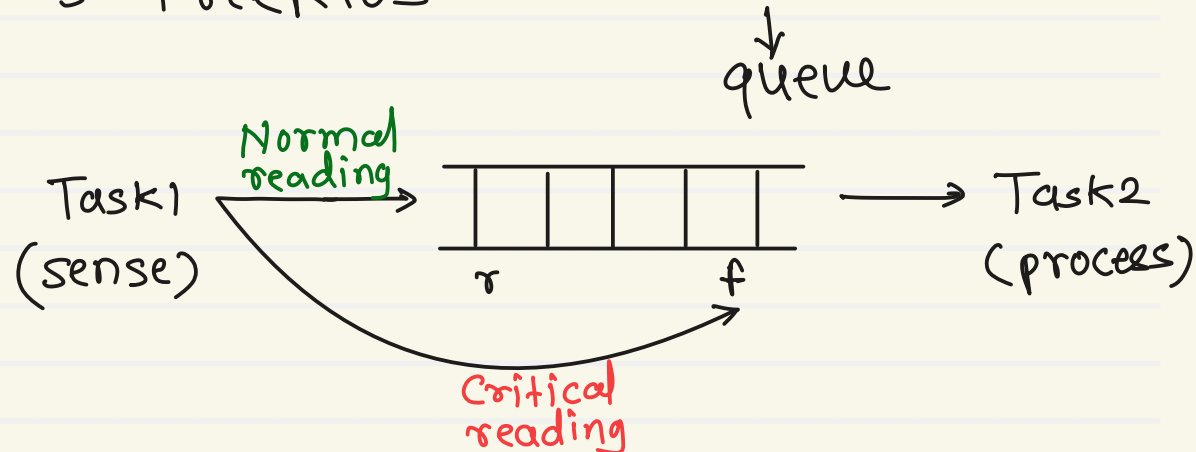
$$s(n) = O(n)$$

3. undo/redo operations

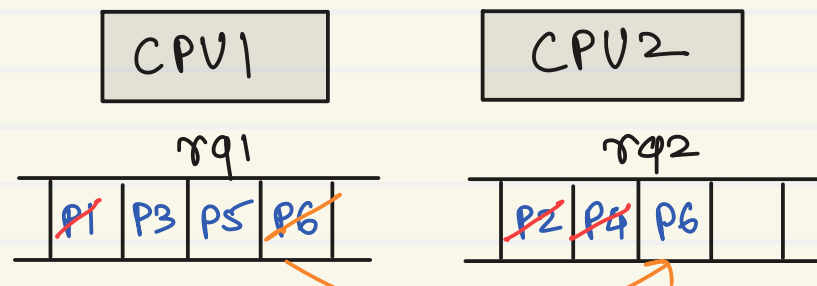


4. browser history

5. FreeRTOS → task communication



6. scheduling in multiprocessor systems



- A Steal Algorithm - process stealing



# Hashing

Array : linear search -  $O(n)$   
           binary search -  $O(\log n)$   
 Linked List : linear search -  $O(n)$

} time will vary  
    not to input

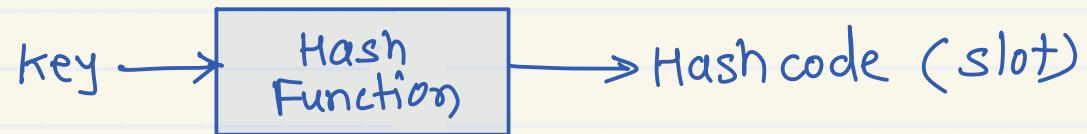
- hashing is a technique in which data can be inserted, deleted and searched in constant average time  $O(1)$
- Implementation of hashing is known as hash table
- Hash table is array of fixed size in which elements are stored in key - value pairs

## Array - Hash table

## Index - Slot

↳ Associative data structure

- In hash table only unique keys are stored
- Every key is mapped with one slot of the table and this is done with the help of mathematical function known as hash function



key → value

88-V1  
93-V2  
100-V3  
54-V4  
76-V5

Size=10

|         |   |
|---------|---|
| 100, V3 | 0 |
|         | 1 |
|         | 2 |
| 93, V2  | 3 |
| 54, V4  | 4 |
|         | 5 |
| 76, V5  | 6 |
|         | 7 |
| 88, V1  | 8 |
|         | 9 |

Hash Table

$$h(k) = k \% \text{size}$$

hash function

$$h(88) = 88 \% 10 = 8 \leftarrow \text{hash code}$$

$$h(93) = 93 \% 10 = 3$$

$$h(100) = 100 \% 10 = 0$$

$$h(54) = 54 \% 10 = 4$$

$$h(76) = 76 \% 10 = 6$$

Add

1. Find slot =  $h(k)$
2.  $arr[slot] = \text{data}$

Search

1. Find slot =  $h(k)$
2. return  $arr[slot].value$

Delete

1. Find slot =  $h(k)$
2.  $arr[slot] = \text{null}$

constant time

$$T(n) = O(1)$$



Size = 10

key → value

88-V1  
93-V2  
100-V3  
54-V4  
76-V5  
113-V6

Ⓢ

|         |   |
|---------|---|
| 100, V3 | 0 |
|         | 1 |
|         | 2 |
| 93, V2  | 3 |
| 54, V4  | 4 |
|         | 5 |
| 76, V5  | 6 |
|         | 7 |
| 88, V1  | 8 |
|         | 9 |

Hash Table

$$h(k) = k \% \text{ size}$$

$$h(88) = 88 \% 10 = 8$$

$$h(93) = 93 \% 10 = 3$$

$$h(100) = 100 \% 10 = 0$$

$$h(54) = 54 \% 10 = 4$$

$$h(76) = 76 \% 10 = 6$$

$$h(113) = 113 \% 10 = 3$$

**Collision:**

When two or more distinct keys yield/give same slot, called as collision.

**Collision handling / resolution techniques:-**

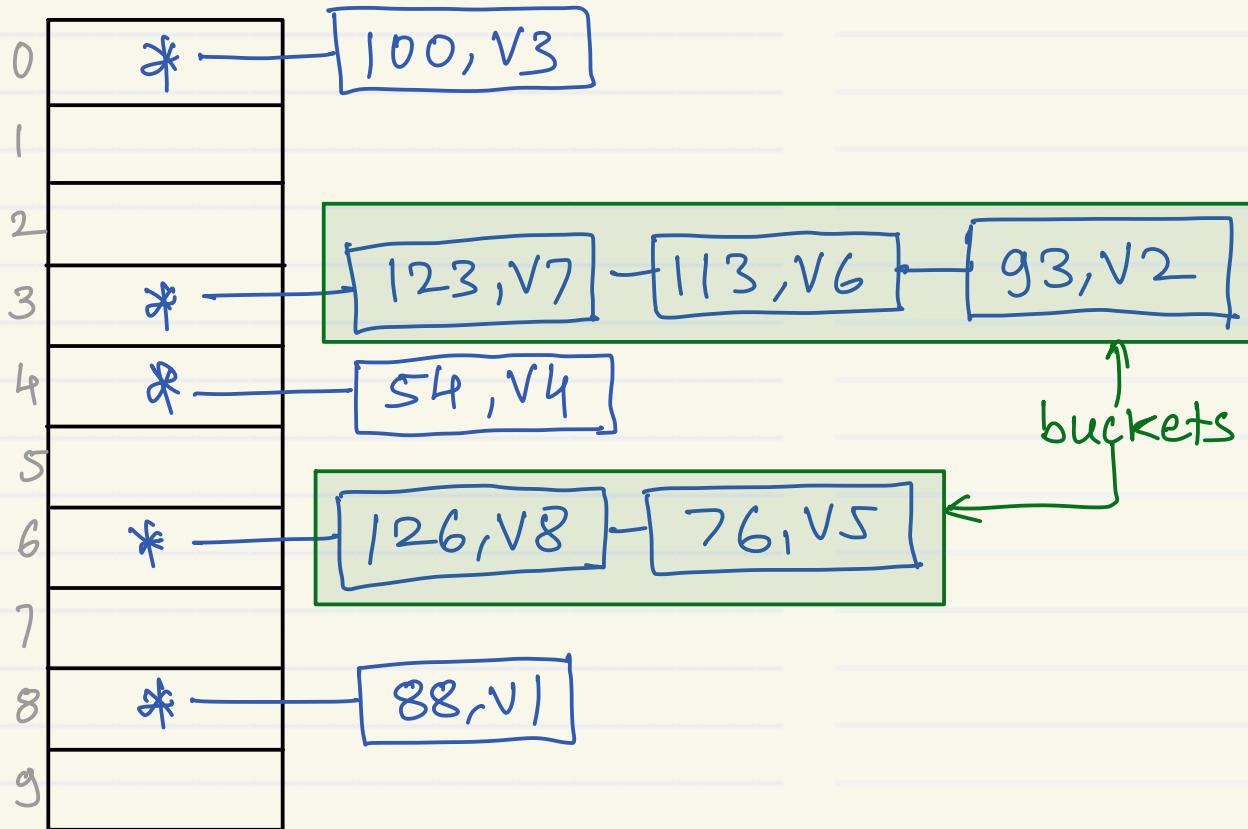
1. Closed Addressing (chaining)
2. Open Addressing
  - i. linear probing
  - ii. quadratic probing
  - iii. double hashing.

# Closed Addressing / Chaining / Separate Chaining

Size = 10 (per slot one linked list is maintained)

$$h(k) = k \% \text{ size}$$

88-V1  
93-V2  
100-V3  
54-V4  
76-V5  
113-V6  
123-V7  
126-V8



Hash Table

**Advantage :**  
can store multiple key value pairs

**Disadvantages:**

- i. key value pairs are stored outside the table
- ii. space requirement is more
- iii. worst case time complexity is  $O(n)$ .  
(if maximum keys yield same slot)